Workgroup:	Network Working Group
Internet-Draft:	draft-narvaneni-agent-uri-04
Published:	20 October 2026
Intended Status:	Standards Track
Expires:	23 April 2027
Authors:	Y. Narvaneni *Independent Researcher*	S. P. Ravi *Independent Researcher*
---	---	---

The agent URI Scheme and Verifiable Agent Identity

Abstract

Software agents increasingly act on the Internet on behalf of people and organizations: they call APIs, browse websites, and delegate work to other agents. Protocols such as A2A and MCP define how agents communicate, but there is no interoperable way to answer a prior question: *which agent is this, and how can its claims be verified?*

This document defines the `agent` URI scheme as a stable, human-readable identifier for an agent, a deterministic mapping from that identifier to an HTTPS-hosted Agent Identity Document, anchoring of the agent's keys to a web origin or a Decentralized Identifier, and signed statements that bind the agent to its protocol metadata. It also defines profiles that let an agent prove its identity on HTTP requests, building on HTTP Message Signatures for automated traffic, and that let short-lived agent instances act under the agent's identity using the WIMSE Workload Identity Token format. It deliberately does not define how agents communicate, describe their capabilities, or are invoked.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 23 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 4
   1.1. Design Principles 5
   1.2. Non-Goals 5
2. Conventions and Terminology 5
3. Overview 6
4. The agent URI Scheme 7
   4.1. Syntax 7
   4.2. Normalization and Comparison 8
   4.3. Dereferencing 8
   4.4. Fragments 9
5. Agent Identity Document 9
   5.1. Format 9
   5.2. Key Identifiers 9
   5.3. Key Purposes 10
   5.4. Links 10
   5.5. Example 11
   5.6. Retrieval and Validation 12
6. Anchoring 13
   6.1. Origin-Anchored Agents 13
   6.2. DID-Anchored Agents 13
   6.3. Lifecycle, Rotation, and Revocation 14
7. Agent Statements 14
   7.1. Verification 15

8. Profile: Request Signing 15
8.1. Signing 15
8.2. Verification 16
9. Profile: Agent Instances 17
10. Profile: Authorization 18
10.1. The Agent as an OAuth Client 18
10.2. Delegation 20
10.3. Protected Resources 20
11. Discovery 20
12. Relationship to Other Protocols 20
13. Security Considerations 21
13.1. Trust Model 21
13.2. Handle Reassignment 21
13.3. Operator Claims 21
13.4. Verifier-Initiated Fetches 22
13.5. Request Content and Replay 22
13.6. Key Compromise and Key Separation 22
13.7. Cross-Origin Metadata 23
13.8. Confusable Names 23
14. Privacy Considerations 23
15. IANA Considerations 23
15.1. URI Scheme Registration 23
15.2. Well-Known URI Registration 24
15.3. Media Types 24
15.4. Signature-Agent Type 24
15.5. Link Relations 24
15.6. OAuth Client Metadata 24
15.7. JSON Web Key Parameters 24
15.8. JSON Web Token Claims 25

16. References 25
16.1. Normative References 25
16.2. Informative References 27
Appendix A. Test Vectors: Agent URI Processing 28
A.1. Parsing and Canonicalization 28
A.2. Mapping to AID URLs 29
A.3. Reverse Mapping of Signature-Agent Values 30
A.4. AID Origin Comparison 30
Appendix B. Open Issues 31
Appendix C. Changes from draft-narvaneni-agent-uri-03 32
Acknowledgments 32
Authors' Addresses 33

1. Introduction

Agents are now first-class actors on the Internet. A single user task can cause an agent to call dozens of APIs and other agents, each of which needs to decide whether to serve the request. Today those services can see a TLS client, an IP address, perhaps an OAuth access token issued to *some* client, and a User-Agent string that anyone can forge. They cannot reliably tell which agent is calling, or whether the protocol metadata it advertises was really published by whoever controls it.

Communication protocols already exist. [A2A] defines agent-to-agent interaction and an Agent Card; [MCP] defines how agents reach tools and publishes server metadata; OpenAPI describes HTTP APIs. Each of these describes *how to talk to* an endpoint. None of them defines a portable answer to *who* an agent is that works across all of them.

This document defines that missing identity layer. Its **core** (Section 4 to Section 7) is:

1. **A name.** The agent URI (e.g., `agent://example.com/planner`) is a stable, human-readable identifier for an agent (Section 4).
2. **A document.** Every agent URI maps deterministically to an HTTPS URL that serves an Agent Identity Document listing the agent's claimed operator, public keys, and links to its protocol metadata (Section 5).
3. **An anchor.** The agent's keys are trusted because of where the document is served (the origin derived from the agent URI) or because of a Decentralized Identifier bound to the agent URI (Section 6).
4. **Statements.** The agent signs statements binding itself to specific protocol metadata, so that metadata can be hosted, mirrored, or indexed anywhere (Section 7).

On top of the core, this document defines **profiles** that are deliberately separable and are expected to track related IETF work:

- a request-signing profile of [WEB-BOT-AUTH] (Section 8);
- an agent-instance profile that reuses the WIMSE Workload Identity Token format [WIMSE-CREDS] with AID-based trust (Section 9); and
- OAuth 2.0 bindings (Section 10).

1.1. Design Principles

- **Identify, do not invoke.** An agent URI names an agent. It is not a locator for an endpoint and carries no invocation parameters.
- **Reuse before invent.** Every mechanism is a profile of an existing or in-progress standard: HTTPS and well-known URIs [RFC8615], JWK [RFC7517], JWS [RFC7515], HTTP Message Signatures [RFC9421] as profiled by [WEB-BOT-AUTH], the WIMSE Workload Identity Token format [WIMSE-CREDS], OAuth 2.0, and DIDs [DID-CORE].
- **Anchor trust outside the document being verified.** A key is trusted because of the origin or DID it is published under, never because a document names it about itself.
- **Separate the name from the long-lived identity when needed.** Domains change hands; agents get renamed and sold. The scheme supports a stable cryptographic identifier behind the human-readable name, with continuity for verifiers that have seen it before (Section 6.2).
- **Authenticate keys, not claims.** Verification establishes control of keys anchored to an agent URI. Everything else an agent says about itself, including who operates it, remains a claim.
- **Work for agents that live for seconds.** Most agent executions are ephemeral. Instances act under the identity of their agent without being individually published.

1.2. Non-Goals

The following are explicitly out of scope:

- Communication semantics, message formats, task lifecycles, and invocation. These belong to A2A, MCP, OpenAPI-described APIs, and similar protocols.
- Capability, skill, or tool description. Protocol metadata documents (for example, an A2A Agent Card or an MCP server card) remain authoritative for these.
- Transport bindings.
- Verification of organizations. This document does not verify that the operator an agent claims is real or actually operates it.
- Search, ranking, or registry services. This document makes identity verifiable so that any registry can index agents without being trusted.
- Reputation, pricing, payments, and service levels.
- Authorization policy. Knowing who an agent is does not grant it anything.

2. Conventions and Terminology

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

This document uses the terms "JSON object", "member", and "string" from [RFC8259], and "Dictionary", "Item", and "String" from [RFC9651].

Agent: A software system that acts autonomously or semi-autonomously, typically on behalf of a person or organization.

Operator: The person or organization that an agent claims is accountable for it.

Agent URI: An `agent` URI as defined in Section 4. The human-readable name of an agent.

Agent Identity Document (AID): A JSON document, retrieved by dereferencing an agent URI, that lists the agent's claimed operator, keys, and links (Section 5).

AID URL: The HTTPS URL that an agent URI maps to (Section 4.3).

AID Origin: The tuple ("https", canonical host, effective port) of the AID URL, where the effective port is 443 when no port is present.

Agent Key: A public key anchored to an agent (Section 6).

Key Identifier: The JWK SHA-256 Thumbprint of an Agent Key (Section 5.2).

Instance: A running, typically short-lived execution of an agent that holds its own key pair and acts under the agent's identity (Section 9).

Verifier: Any party that checks an agent's identity: a website or API receiving requests, an authorization server, a registry, or another agent.

3. Overview

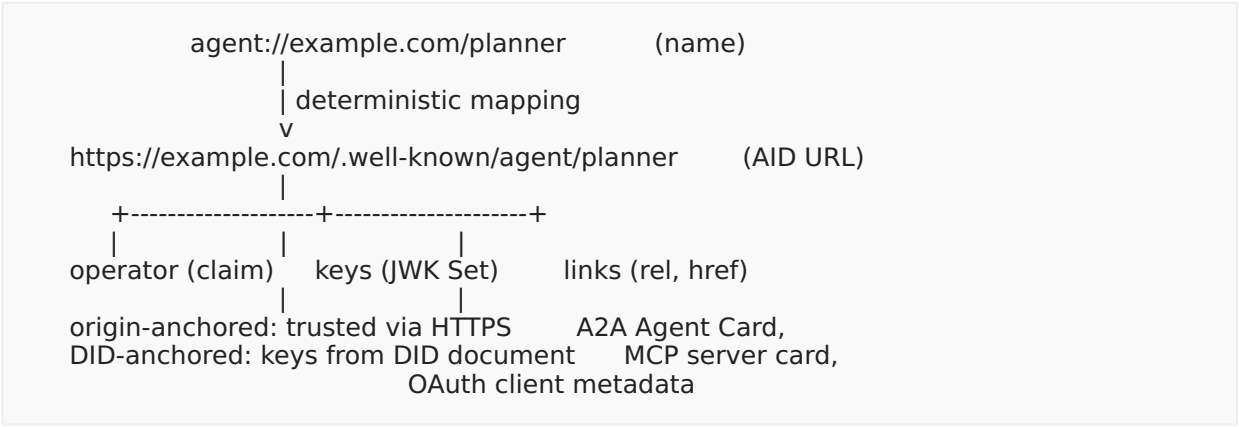


Figure 1: From agent URI to verifiable identity

A typical verification, for a request received by an API:

1. The request carries an HTTP message signature and a `Signature-Agent` member whose value is the AID URL of `agent://example.com/planner`.

2. The verifier derives the agent URI from the AID URL and retrieves the AID (or uses a cached copy).
3. The verifier finds the anchored key identified by the signature and verifies the signature and the request content.
4. The verifier now knows that the request was made by a holder of a key anchored to agent://example.com/planner. It also has the operator the agent *claims*, which it must not treat as verified (Section 13.3). Whether to serve the request remains a policy decision.

4. The agent URI Scheme

4.1. Syntax

The following ABNF [RFC5234] uses fragment from [RFC3986] and ALPHA and DIGIT from [RFC5234].

```
agent-uri    = "agent://" agent-host [ name-path ] [ "#" fragment ]

agent-host   = dns-name [ ":" port ]
dns-name     = label 1*( "." label )
label        = let-dig [ *61( let-dig / "-" ) let-dig ]
let-dig      = ALPHA / DIGIT
port         = %x31-39 *4DIGIT        ; 1-65535, no leading zero

name-path    = 1*8( "/" name-segment )
name-segment = lc-let-dig [ *61( lc-let-dig / "-" ) lc-let-dig ]
lc-let-dig   = %x61-7A / DIGIT        ; a-z, 0-9
```

Figure 2: ABNF for the agent URI

Additional constraints:

- dns-name **MUST** contain at least two labels (as required by the ABNF), **MUST NOT** exceed 253 octets, and its final label **MUST** contain at least one ALPHA character. This excludes IPv4 addresses in dotted-decimal form and single-label names such as localhost.
- Internationalized domain names **MUST** be represented as A-labels [RFC5890] [RFC5891]. IP literals and userinfo are not permitted.
- port **MUST NOT** exceed 65535.
- An agent URI **MUST NOT** contain a query component.
- An agent URI with an empty name-path (e.g., agent://planner.example.com) identifies the domain's root agent.

Examples:

- agent://example.com/planner
- agent://planner.example.com
- agent://example.com/acme/billing-assistant
- agent://example.com/planner#kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k (a reference to one of the agent's keys; see Section 4.4)

4.2. Normalization and Comparison

The canonical form of an agent URI is produced by:

1. Lowercasing the scheme and dns-name.
2. Converting any U-labels to A-labels.
3. Removing a port equal to 443, and removing a ":" followed by an empty port.
4. Removing a single trailing "/" if present.

Parsers **MAY** accept input that differs from the ABNF only by the deviations corrected above, and **MUST** normalize it. Any other deviation (uppercase in name-path, percent-encoding, dot-segments, a query, a trailing "." on the host) **MUST** be rejected.

Two agent URIs identify the same agent if and only if their canonical forms, with any fragment removed, are identical octet strings. In particular, agent://example.com/planner and agent://planner.example.com are different agents, even if the same operator runs both.

Agent URIs in AIDs, statements, credentials, and tokens **MUST** be in canonical form and **MUST NOT** contain a fragment, unless a fragment is explicitly permitted.

4.3. Dereferencing

An agent URI is dereferenced by mapping it to its AID URL:

```
https://<agent-host>/.well-known/agent<name-path>      (named agent)
https://<agent-host>/.well-known/agent/_               (root agent)
```

For example:

Agent URI	AID URL
agent://example.com/planner	https://example.com/.well-known/agent/planner
agent://example.com/acme/billing-assistant	https://example.com/.well-known/agent/acme/billing-assistant
agent://planner.example.com	https://planner.example.com/.well-known/agent/_
agent://example.com:8443/planner	https://example.com:8443/.well-known/agent/planner

Table 1: Agent URI to AID URL mapping

The root agent uses the reserved segment _, which cannot appear in a name-segment. This allows the root AID and named AIDs to coexist on static file hosting.

The mapping is purely syntactic and is reversible: an HTTPS URL is an AID URL if and only if it has no query or fragment and applying the reverse mapping yields an agent URI that is valid and canonical. No registry or index needs to be fetched to dereference an agent URI.

4.4. Fragments

A fragment in an agent URI identifies an Agent Key by its Key Identifier (Section 5.2). No other fragment semantics are defined.

5. Agent Identity Document

5.1. Format

An AID is a JSON object [RFC8259] served with media type application/agent-id+json (Section 15.3). It has the following members:

id: **REQUIRED**. String. The canonical agent URI that this document describes.

operator: **REQUIRED**. Object stating who the agent claims is accountable for it, with members name (**REQUIRED**, string) and url (**OPTIONAL**, string, HTTPS URL). These values are self-asserted (Section 13.3).

keys: **REQUIRED** for origin-anchored agents; **MUST** be absent for DID-anchored agents. A JWK Set [RFC7517] containing the agent's public keys (Section 6.1).

did: **OPTIONAL**. String. A DID [DID-CORE] whose DID document is the anchor for this agent's keys (Section 6.2).

name: **OPTIONAL**. String. A display name for people. It has no identity semantics.

status: **OPTIONAL**. String. One of "active" (default), "suspended", or "retired" (Section 6.3).

links: **OPTIONAL**. Array of link objects (Section 5.4).

updated: **OPTIONAL**. String. The time the document was last changed, as an RFC 3339 [RFC3339] timestamp.

Members not defined here **MUST** be ignored by verifiers that do not understand them. Extension member names **SHOULD** be namespaced (e.g., "acme_tier") to avoid collision with future versions of this document.

An AID **MUST NOT** exceed 64 KiB.

5.2. Key Identifiers

Every Agent Key is identified by its **Key Identifier**: the base64url-encoded JWK SHA-256 Thumbprint [RFC7638] of the key, computed as specified in [RFC7638] for EC keys and in Appendix A.3 of [RFC8037] for OKP keys. This is the same identifier [WEB-BOT-AUTH] uses for keyid.

Key Identifiers are used everywhere a key is named in this document: in the kid of JWKs in an AID, in agent URI fragments, in the JWS kid header of statements and Workload Identity Tokens, and in the keyid signature parameter of request signatures.

Verifiers **MUST** compute the thumbprint of each candidate key themselves and select keys by comparing it with the presented Key Identifier. They **MUST NOT** rely on an operator-supplied `kid` or DID verification method `id` to select a key.

For DID-anchored agents, each verification method is converted to a JWK before the thumbprint is computed:

- a `publicKeyJwk` value is used as-is (private members **MUST NOT** be present);
- a `publicKeyMultibase` value that is a Multikey encoding of an Ed25519 public key (base58btc with the `0xed01` header, as defined in [CID]) is converted to an OKP JWK with `"crv": "Ed25519"`; and
- verification methods in any other representation **MAY** be supported; those a verifier cannot convert are ignored.

5.3. Key Purposes

Agent Keys are used for two purposes:

Assertion: Signing Agent Statements (Section 7) and Workload Identity Tokens for instances (Section 9).

Authentication: Signing HTTP requests directly as the agent (Section 8).

For DID-anchored agents, a key is usable for assertion only if it is referenced from the DID document's `assertionMethod` relationship, and for authentication only if it is referenced from `authentication` [DID-CORE]. Verifiers **MUST** enforce these relationships.

For origin-anchored agents, a JWK in `keys` **MAY** carry an `agent_use` member (Section 15.7): an array containing one or both of the strings `"assertion"` and `"authentication"`. A key is usable only for the purposes listed; a key without `agent_use` is usable for both. Verifiers **MUST** enforce `agent_use`. A key whose `agent_use` is not an array of strings (for example, a bare string), or contains no recognized value, **MUST** be ignored; it is never treated as unrestricted. An AID in which two JWKs have the same Key Identifier **MUST** be rejected, so that purposes cannot be combined across entries for the same key material. Operators **SHOULD** give every key a single purpose, so that a key held by a running agent for request signing cannot mint Workload Identity Tokens or sign statements (Section 13.6).

5.4. Links

Each link object has the following members, with the semantics of Web Linking [RFC8288]:

`rel`: **REQUIRED**. String. A registered link relation type or a URI.

`href`: **REQUIRED**. String. An absolute HTTPS URL.

`type`: **OPTIONAL**. String. The media type of the target.

`statement`: **OPTIONAL**. String. An Agent Statement (Section 7) about the target.

Link relations expected to be common include:

Relation	Target
agent-card	The agent's [A2A] Agent Card
mcp-server-card	The agent's [MCP] server metadata
service-desc	An OpenAPI or other API description [RFC9727]
oauth-client-metadata	The agent's OAuth Client ID Metadata Document (Section 10.1)

Table 2: Common link relations

A link whose href has the **same origin as the AID Origin** is trusted to the same degree as the AID itself. To compare, the verifier lowercases the href host, converts it to A-labels, and uses port 443 if none is present; the result must equal the AID Origin exactly (Appendix A.4). No other normalization is applied; in particular, a trailing "." is not removed from the host. A link to any other origin **MUST** carry a statement, and verifiers **MUST** ignore cross-origin links without a valid statement (Section 7.1). Because verifier-initiated fetches never follow redirects (Section 13.4), a same-origin href cannot acquire the same-origin exemption for content served from another origin.

For DID-anchored agents the same-origin exemption does not apply: every link **MUST** carry a statement signed with a key from the DID document's assertionMethod, and verifiers **MUST** ignore links without one. Otherwise whoever controls the origin could replace linked metadata, such as an OAuth client metadata document containing client keys, without holding any key authorized by the DID (Section 13.1).

5.5. Example

```
{
 "id": "agent://example.com/planner",
 "name": "Example Trip Planner",
 "operator": {
  "name": "Example Travel Ltd",
  "url": "https://example.com"
 },
 "keys": {
  "keys": [
   {
    "kty": "OKP",
    "crv": "Ed25519",
    "x": "11qYAYKxCrfVS_7TyWQHOg7hcvPapiMlrwIaaPcHURo",
    "kid": "kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k",
    "alg": "EdDSA",
    "use": "sig"
   }
  ]
 },
 "links": [
  {
   "rel": "agent-card",
   "href": "https://example.com/planner/agent-card.json",
   "type": "application/json"
  },
  {
   "rel": "mcp-server-card",
   "href": "https://tools.example-cdn.net/planner/server.json",
   "statement": "eyJ0eXAiOiJhZ2VudC1zdGF0ZW1lbnQrand0Ii..."
  },
  {
   "rel": "oauth-client-metadata",
   "href": "https://example.com/planner/oauth-client.json"
  }
 ],
 "updated": "2026-10-20T09:00:00Z"
}
```

Figure 3: Example Agent Identity Document (origin-anchored)

5.6. Retrieval and Validation

To obtain the AID for an agent URI, a verifier:

1. Parses and canonicalizes the agent URI (Section 4.2), rejecting it on failure.
2. Constructs the AID URL (Section 4.3).
3. Applies the fetch policy of Section 13.4.
4. Sends an HTTPS GET with `Accept: application/agent-id+json`, validating the server certificate for dns-name.
5. Treats any status code other than 200 (OK) [RFC9110], and other than 304 (Not Modified) received in response to a conditional request revalidating a cached copy [RFC9111], as a retrieval failure. Redirects are always failures.

6. Rejects responses larger than 64 KiB, or that are not valid JSON objects. Verifiers **MUST NOT** reject a response solely because its media type is application/json (to support static hosting) but **MUST** reject any other media type.
7. Rejects the document unless its id member is identical to the canonical agent URI.
8. Performs the anchoring checks of Section 6, including the DID binding and pinning checks of Section 6.2 when applicable.

Verifiers **SHOULD** cache AIDs according to HTTP caching [RFC9111]. Operators **SHOULD NOT** set a freshness lifetime longer than 24 hours, so that key removal takes effect promptly (Section 6.3). Verifiers **SHOULD** cache failures for a short period to limit repeated fetches.

6. Anchoring

An Agent Key is *anchored* when the verifier has established that whoever controls the agent URI published it. This document defines two anchoring modes. An AID uses exactly one.

6.1. Origin-Anchored Agents

The AID contains keys and no did. The keys are anchored by the HTTPS retrieval of Section 5.6: control of the AID Origin is control of the agent's identity.

Each JWK in keys **MUST**:

- contain only public key parameters;
- have a kid equal to its Key Identifier (Section 5.2);
- have an alg member; and
- be usable for signatures ("use": "sig", or a key_ops containing "verify").

Implementations **MUST** support EdDSA (Ed25519) and ES256. Algorithms **MUST** be restricted as described in [RFC8725]; in particular, none and symmetric algorithms **MUST NOT** be accepted.

This mode needs nothing beyond a web server. Its weakness is that identity is exactly as durable as control of the domain (Section 13.2).

6.2. DID-Anchored Agents

The AID contains a did member and no keys. The agent's keys are the verification methods of the DID document, with the purposes defined in Section 5.3.

The binding between the agent URI and the DID **MUST** be bidirectional:

- the AID retrieved for the agent URI names the DID in did; and
- the DID document lists the canonical agent URI in its alsoKnownAs property [DID-CORE].

Verifiers **MUST** check both directions and **MUST** treat the agent as unverified if either is missing.

Pinning. A verifier that has previously verified an agent URI as DID-anchored and recorded its DID **MUST NOT** attribute a later AID for that agent URI to the same agent unless it names the same DID. An AID that is origin-anchored or names a different DID **MUST** be treated as a different identity (for example, by rejecting it, or by treating it as a new agent with no history), not as a key rotation. Verifiers that record identities **SHOULD** record the DID.

Continuity and its limits. For a verifier that has pinned the DID, DID anchoring preserves identity across a rename or domain move: the operator publishes an AID under the new agent URI and adds that URI to alsoKnownAs. For a verifier with no prior state, DID anchoring gives no more assurance than origin anchoring, because whoever controls the agent URI's origin can publish an origin-anchored AID instead (Section 13.1).

DID methods. A DID whose resolution depends on the same host as the agent URI (for example, did:web on that host) adds no independence from the origin. Operators **SHOULD NOT** use such a DID for DID anchoring, and verifiers **MUST NOT** treat it as providing assurance beyond origin anchoring. DID methods that support verifiable key-rotation history (for example, [DID-WEBVH] hosted independently of the agent URI's host) are **RECOMMENDED**. Verifiers **MUST** maintain an allowlist of DID methods they support and **MUST** reject any other method. Network requests made by DID resolvers are subject to Section 13.4.

6.3. Lifecycle, Rotation, and Revocation

Removing a key from the AID (or DID document) revokes it: every statement and Workload Identity Token it signed stops verifying once verifiers' cached copies expire. Rotation and revocation are therefore the same operation, and operators rotate as follows:

1. Publish the new key alongside the old one.
2. Re-issue all statements and credentials signed by the old key, including statements carried in AIDs copied by mirrors and registries.
3. Remove the old key once re-issued statements have propagated.

Statements about metadata that changes **SHOULD** carry an `exp` so that copies held by third parties are refreshed regularly.

If status is `"suspended"` or `"retired"`, verifiers **MUST** treat all signatures, statements, and credentials from the agent as invalid. A retired agent URI **SHOULD NOT** be reassigned to a different agent.

Operators that need auditable key history **MAY** additionally record their AIDs and statements in a transparency service [RFC9943]. This document does not require it.

7. Agent Statements

An Agent Statement is a JWS [RFC7515] in compact serialization, signed with an Agent Key usable for assertion, that binds the agent to a specific version of a resource such as an Agent Card. It allows protocol metadata to be served from any origin, mirrored, or included in registries without losing its link to the agent.

The JWS Protected Header **MUST** contain:

- typ: `"agent-statement+jwt"`;
- alg: the algorithm of the signing key;

- kid: the Key Identifier of the signing key (Section 5.2).

The payload is a JWT Claims Set [RFC7519] with:

iss: **REQUIRED**. The canonical agent URI, without a fragment.

sub: **REQUIRED**. The URL of the resource the statement is about.

resource_digest: **REQUIRED**. An object with members alg ("sha-256"), canon ("jcs" or "none"), and value (the base64url-encoded digest). With "jcs", the digest is computed over the resource canonicalized per [RFC8785]; with "none", over the exact bytes of the representation.

iat: **REQUIRED**. Issue time.

exp: **OPTIONAL**. Expiry time.

7.1. Verification

To verify a statement carried in a link of an AID, a verifier:

1. Checks that typ is "agent-statement+jwt".
2. Checks that iss is identical to the AID's id, and checks the AID's status (Section 6.3).
3. Checks that sub is identical to the link's href. A statement whose sub differs **MUST** be rejected, even if it is otherwise valid.
4. Selects an anchored key usable for assertion whose Key Identifier equals kid, and verifies the signature.
5. Checks iat and, if present, exp.
6. Obtains the resource at sub, either by fetching it under the policy of Section 13.4 or by using a copy, and compares its digest with resource_digest.

Statements obtained other than through an AID link (for example, from a registry) are verified in the same way, except that step 2 uses the AID retrieved for iss and step 3 compares sub with the URL the verifier intends to use.

A statement protects only the bytes of the resource it names. It does not extend to other resources that resource refers to, such as a jwks_uri in OAuth client metadata: each such resource needs its own statement before it can be relied on with the same assurance.

Protocol metadata documents that support extension fields **SHOULD** include a reference back to the agent URI, so that a client starting from the metadata can find the identity. How that reference is expressed is defined by each protocol (see Appendix B).

8. Profile: Request Signing

8.1. Signing

An agent proves its identity on an HTTP request by signing it as specified in [WEB-BOT-AUTH], with the additional requirements in this section. A request signed this way is a conforming [WEB-BOT-AUTH] request; this profile adds a new Signature-Agent member type and constrains the signature further.

The request **MUST** include a Signature-Agent Dictionary member, keyed by the signature label, whose value is the AID URL derived from the agent's canonical agent URI (Section 4.3) and which carries the parameter type=agent-id:

```
Signature-Agent: sig1="https://example.com/.well-known/agent/planner"
    ;type=agent-id
```

Figure 4: Signature-Agent member (line break for display only)

The signature **MUST**:

- cover @method, @authority, @target-uri, and "signature-agent";key="<label>";
- cover content-digest [RFC9530] when the request has content;
- cover workload-identity-token when that field is present (Section 9);
- include the created and expires parameters, with expires no more than 300 seconds after created;
- include a keyid parameter whose value is the Key Identifier of the signing key; and
- include the parameter tag="web-bot-auth".

A nonce parameter **SHOULD** be included where the verifier is expected to detect replay.

Example:

```
POST /v1/bookings HTTP/1.1
Host: api.example.net
Content-Type: application/json
Content-Digest: sha-256=:...:
Signature-Agent: sig1="https://example.com/.well-known/agent/planner"
    ;type=agent-id
Signature-Input: sig1=("@method" "@authority" "@target-uri"
    "signature-agent";key="sig1" "content-digest");
    created=1792486800;expires=1792486860;
    keyid="kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k";
    tag="web-bot-auth"
Signature: sig1=:...:
```

Figure 5: A signed request (line breaks for display only)

8.2. Verification

A verifier **MUST** perform all of the following steps, and **MUST NOT** rely on a generic HTTP Message Signatures implementation to enforce them:

1. Select the signature with tag="web-bot-auth" whose label has a Signature-Agent member with type=agent-id. If there is none, the request does not carry an agent identity under this profile.
2. Check that the member value is an AID URL and derive the agent URI from it (Section 4.3). Verifiers **MUST NOT** normalize the value before applying the reverse mapping; a value that is not exactly the AID URL of a canonical agent URI is rejected.

3. Check that the covered components include every component required by Section 8.1, and that expires is no more than 300 seconds after created and the current time is between them, allowing for modest clock skew.
4. Retrieve the AID (Section 5.6) and check status (Section 6.3).
5. If a Workload-Identity-Token field is present, validate it as described in Section 9; the verification key is then the key in its cnf claim, and keyid **MUST** equal that key's Key Identifier. Otherwise, select an anchored key usable for authentication whose Key Identifier equals keyid. Key lookup is keyed on the pair (agent URI, keyid), as required by [WEB-BOT-AUTH].
6. Verify the signature as specified in [RFC9421].
7. If the request has content, recompute the digest of the received content for a supported algorithm as specified in [RFC9530] and compare it with the covered Content-Digest value. A request with content and a missing, unsupported, or mismatched Content-Digest **MUST** be rejected. A valid signature over Content-Digest does not by itself authenticate the content (Section 7.2.8 of [RFC9421]).

On success, the verifier has established that the request, including its content, was made by a holder of a key anchored to the agent URI (directly, or through an instance credential). On failure, the verifier **MUST NOT** treat the request as coming from the named agent; it **MAY** process it as an unauthenticated request.

A [WEB-BOT-AUTH] verifier that does not support type=agent-id ignores the member, as that specification requires, and so treats the request as not signed by a known agent rather than failing.

9. Profile: Agent Instances

Agents frequently run as many short-lived instances, for example one per task. Publishing a key for each instance in the AID does not scale, and placing the agent's long-term private keys in every instance is unsafe.

Instead, the agent's controlling infrastructure issues each instance a token in the WIMSE Workload Identity Token (WIT) format [WIMSE-CREDS] that binds the instance's key to the agent.

This profile reuses the WIT format and header field but not the WIMSE trust model. Section 3 of [WIMSE-CREDS] requires verifiers to bind each trust domain to issuers and trust anchors provisioned out of band. This profile replaces that requirement: the only authorized issuers of a WIT whose sub is an agent URI are the assertion keys anchored to *exactly that agent URI* (Section 6), discovered from its AID. Verifiers that also implement WIMSE **MUST NOT** treat these keys as trust anchors for any other workload identifier, in the same trust domain or otherwise, and **MUST NOT** accept such a WIT under WIMSE trust-domain configuration alone.

This profile applies only to agent URIs without a port, because WIMSE workload identifiers cannot contain one. An agent whose URI has a port **MUST NOT** issue WITs under this profile, and verifiers **MUST** reject a WIT whose sub has a port. A port **MUST NOT** be removed to make a URI fit, since that names a different agent.

The WIT **MUST** conform to the format and claim requirements of Section 5.1 of [WIMSE-CREDS] and additionally:

- is signed with an Agent Key usable for assertion, with the JWS kid header set to that key's Key Identifier;
- has a cnf claim [RFC7800] containing the instance's public key as a jwk, as required by [WIMSE-CREDS];
- has sub set to the canonical agent URI;
- has iss, if present, set to the canonical agent URI;
- has a jti claim, which **MUST** be generated at random and not reused (Section 14);
- has an iat claim that is not in the future; and
- has an exp later than iat and no more than one hour after it.

The instance signs requests with its own key as in Section 8.1, with keyid set to the Key Identifier of the key in the WIT's cnf.jwk, and sends the WIT in the Workload-Identity-Token field defined by [WIMSE-CREDS], which the signature covers.

To validate the WIT, the verifier:

1. Validates the WIT's format and claims as specified in Section 5.1 of [WIMSE-CREDS] and the additional requirements above. The verifier **MUST** reject the WIT if iat is missing; if iat is later than the current time plus the allowed clock skew; unless 0 < exp - iat <= 3600; if exp has passed; if nbf is present and has not yet been reached; or if sub has a port. These checks **MUST** be performed explicitly, because [RFC7519] does not require JWT libraries to reject a future iat.
2. Checks that sub is identical to the agent URI derived from Signature-Agent, and retrieves its AID; trust-anchor provisioning is replaced as described above.
3. Selects an anchored key usable for assertion whose Key Identifier equals the WIT's kid, and verifies the WIT signature.
4. Checks that the request signature's keyid equals the Key Identifier of cnf.jwk, and verifies the request signature with that key using the algorithm in cnf.jwk.alg.

This profile does not use the WIMSE HTTP signature profile [WIMSE-HTTPSIG], whose tag and covered components differ; see Appendix B.

10. Profile: Authorization

Identity is not authorization. This section connects agent identity to OAuth 2.0 without defining new token formats.

10.1. The Agent as an OAuth Client

An agent that obtains OAuth tokens **SHOULD** use a Client ID Metadata Document [OAUTH-CIMD] as its client identifier and link to it from its AID with rel="oauth-client-metadata". The client metadata document **SHOULD** include the agent_uri client metadata parameter (Section 15.6) containing the canonical agent URI.

The agent_uri value is a claim made by whoever publishes the client metadata document. Ordinary client authentication [OAUTH-CIMD] does not authenticate it: anyone can publish client metadata that names someone else's agent URI and then authenticate correctly as that client.

Association check. The association between a client_id and an agent is verified by retrieving the AID for the claimed agent URI (Section 5.6), including all anchoring and status checks, and confirming that the AID has an oauth-client-metadata link whose href is identical to the client_id, with any statement that link requires (Section 5.4). An authorization server that does not attribute agent identity **MAY** skip this check. On its own, the check does not show that the party making the current request controls the agent, because a public client_id can be used by anyone.

Agent attribution. An authorization server **MUST NOT** treat the current caller as an authenticated agent unless one of the following holds:

- the association check succeeds for the exact client_id, **and** the client authenticates with private_key_jwt in the current exchange using qualifying client keys (below); or
- a request signature for the agent URI is verified under Section 8 in the current exchange.

A valid client assertion without a successful association check leaves the agent identity unverified.

Qualifying client keys. A statement protects only the bytes it names (Section 7), so a statement over the client metadata document protects a jwks_uri value but not the keys served there. For the private_key_jwt route, client keys qualify only if they are:

- embedded in the client metadata document's jwks, where that document is served from the AID Origin (origin-anchored agents only) or is covered by a valid statement (Section 5.4); or
- served from a jwks_uri whose origin, compared as in Section 5.4, equals the AID Origin (origin-anchored agents only).

For DID-anchored agents, only embedded jwks in a statement-covered client metadata document qualify; no jwks_uri qualifies, even on the AID Origin. Agents whose client keys do not qualify **MUST** use request signatures for agent attribution.

Which agent is attributed. The attributed agent is the one whose identity was verified: for the private_key_jwt route, the agent URI that passed the association check; for the request-signature route, the agent URI derived from Signature-Agent. If the client metadata document's agent_uri is absent or names a different agent, the authorization server **MUST NOT** display or record that value as the agent's identity, and **MAY** reject the request.

Consent. In flows where the user's browser reaches the authorization endpoint through a front-channel redirect, the agent cannot sign that request, and the token request happens only after consent. A consent screen can therefore show a verified agent identity only if attribution succeeded on a back-channel request made for the same authorization request before consent. Agents that want verified identity shown at consent **SHOULD** use Pushed Authorization Requests [RFC9126], with the pushed request either signed under Section 8 or authenticated with private_key_jwt satisfying the attribution rule above. Authorization servers **SHOULD** support this.

Otherwise, the authorization server **MUST** handle the client as it would any client using that client_id, and **MUST NOT** present it to users as a verified agent identity. When a verified identity is shown, consent screens **SHOULD** display the attributed agent URI or its DID and **MUST** present the operator name as claimed by the agent (Section 13.3).

10.2. Delegation

When an agent acts on behalf of a user or another agent, the delegation **SHOULD** be expressed with OAuth 2.0 Token Exchange [RFC8693], with the agent identified in the `act` claim by `"sub"` set to its canonical agent URI. As required by [RFC8693], only the current actor is relevant to access-control decisions; nested actors are informational.

Access tokens used by agents **SHOULD** be sender-constrained, for example with DPoP [RFC9449]. The DPoP key **SHOULD** be an instance key (Section 9) or a key dedicated to DPoP. Agent Keys used for assertion **MUST NOT** be used as DPoP keys, both to keep long-term keys out of running agents (Section 13.6) and to avoid using one key across JWS statements, HTTP message signatures, and DPoP proofs. Within a single trust domain, [TXN-TOKENS] can carry the agent identity and call context between services.

10.3. Protected Resources

Resource servers advertise their requirements using OAuth 2.0 Protected Resource Metadata [RFC9728]. A resource server **MAY** require a valid agent signature (Section 8) in addition to an access token, and **MAY** apply policy based on the agent URI or its DID.

11. Discovery

Dereferencing a known agent URI requires no discovery. To let others find which agents a domain operates, a domain **MAY** list them in an API catalog [RFC9727] (a linkset [RFC9264]) at `/.well-known/api-catalog`, using the agent URI as the link context and `agent-identity` links to the AIDs:

```
{
 "linkset": [
  {
   "anchor": "agent://example.com/planner",
   "agent-identity": [
    { "href": "https://example.com/.well-known/agent/planner",
     "type": "application/agent-id+json" }
   ]
  }
 ]
}
```

Figure 6: Listing agents in an API catalog

Publishing a catalog is optional (Section 14). Registries and search services that index agents **SHOULD** verify AIDs and statements themselves rather than asking clients to trust the index.

12. Relationship to Other Protocols

A2A:

An Agent Card describes how to communicate with an agent. The AID links to it with `rel="agent-card"`. A2A clients can verify a card by checking the agent's statement about it.

MCP: MCP server metadata describes tools and how to reach them. The AID links to it with `rel="mcp-server-card"`. MCP clients can identify the calling agent to MCP servers using request signatures.

OpenAPI: The AID links to API descriptions with `rel="service-desc"` [RFC9727].

Web Bot Auth: [WEB-BOT-AUTH] lets automated clients sign requests and resolve keys from an HTTPS URL. Section 8 is a profile of it that adds a `Signature-Agent` type resolving to an AID, which gives the signer a stable agent name, purpose-scoped keys, and an optional DID anchor.

WIMSE: [WIMSE-CREDS] defines workload credentials. Section 9 reuses the WIMSE Workload Identity Token format, with the agent URI as the workload identifier and the AID, rather than out-of-band trust anchors, as the source of the issuer's keys.

13. Security Considerations

13.1. Trust Model

For origin-anchored agents, the trust anchors are DNS and the Web PKI: anyone who can obtain a certificate for the agent URI's host and serve content there can impersonate the agent. This is the same trust model as the web itself and is appropriate for most agents.

DID anchoring reduces this dependency only for verifiers that have pinned the agent's DID (Section 6.2). On first contact, a verifier cannot tell a legitimate origin-anchored agent from an attacker who has taken over the domain and replaced a DID-anchored AID with an origin-anchored one. Even for a pinned DID, assurance holds only if verifiers rely solely on DID-authorized keys and on metadata covered by statements made with them (Section 5.4).

13.2. Handle Reassignment

If a domain lapses and is registered by someone else, the new holder can publish an AID for the same agent URIs. Verifiers that record identities for audit or reputation **SHOULD** record the Key Identifiers (and DID, when present) alongside the agent URI, and **SHOULD** treat a change of all keys for an agent as a significant event.

13.3. Operator Claims

Verification establishes possession of a key anchored to an agent URI or DID. It does not establish who operates the agent: `operator.name` and `operator.url` are supplied by the same party that publishes the AID. The owner of `attacker.example` can claim `"Example Travel Ltd"` as operator and point `operator.url` at the real company's website.

User interfaces and logs **MUST NOT** present an operator name as verified unless it has been confirmed by a separate mechanism trusted by the verifier, which this document does not define. They **SHOULD** display the authenticated agent URI host (in A-label form) or DID alongside any operator name.

13.4. Verifier-Initiated Fetches

Verification causes network requests to destinations chosen by untrusted parties: AID URLs from Signature-Agent values, statement subjects and link targets, and requests made by DID resolvers. Each of these can be used for server-side request forgery or denial of service. The following policy applies to **every** fetch made in the course of verification:

- Only HTTPS URLs are fetched, with certificate validation.
- Destinations whose host is, or resolves to, an address that the IANA IPv4 or IPv6 Special-Purpose Address Registries [RFC6890] mark as not globally reachable **MUST NOT** be fetched. Hosts that are special-use names that do not resolve in the global DNS (for example, localhost or names under local or internal) **MUST NOT** be fetched. Administrators **MAY** configure explicit exceptions for private deployments.
- The resolved address **MUST** be checked and then used for the connection (pinned), so that DNS changes cannot redirect the request after the check.
- HTTP redirects **MUST NOT** be followed, as in [WEB-BOT-AUTH]. Any status other than 200, and other than 304 received in response to a conditional revalidation request, is a failure.
- Verifiers **MUST** enforce connection and response timeouts, and **MUST** bound response sizes: 64 KiB for AIDs and a locally configured limit for other resources.
- Verifiers **MUST** bound the number of fetches per verification. A limit of 4 (for example, the AID, a DID resolution, and a statement subject) is **RECOMMENDED**. A DID resolution counts as one fetch toward this limit; verifiers **MUST** separately bound the requests a DID resolver makes, by configuration for each allowed DID method. Verifiers **MUST NOT** recursively follow links or depends-style references from fetched documents.
- Verifiers **MUST** rate-limit fetches per host and cache results, including failures.

A statement whose signature is valid does not make its sub safe to fetch: an attacker can publish a legitimate AID under its own domain and sign a statement whose sub names an internal address.

13.5. Request Content and Replay

HTTP message signatures authenticate header values, not content. Verifiers **MUST** recompute Content-Digest over received content (Section 8.2); otherwise an attacker can change the content of a signed request while keeping the signed digest.

Signatures cover the method, target URI, and authority, which prevents replay against a different resource or service. The short created-expires window limits replay against the same resource; verifiers for non-idempotent operations **SHOULD** require and track nonce values.

13.6. Key Compromise and Key Separation

Compromise of an Agent Key lets the attacker act as the agent until the key is removed and caches expire. Operators **SHOULD** keep Agent Keys in hardware or a key management service and use instance credentials (Section 9) so that long-term keys are never present in running agents. Workload Identity Tokens **SHOULD** be short-lived, because they cannot be individually revoked.

Operators **SHOULD** use different keys for assertion and authentication, using `agent_use` for origin-anchored keys and DID verification relationships for DID-anchored keys (Section 5.3), and **MUST NOT** use Agent Keys as DPoP keys (Section 10.2).

13.7. Cross-Origin Metadata

Requiring statements bound to the exact `href` for cross-origin links (Section 5.4, Section 7.1) prevents a compromised or malicious third-party host, mirror, or registry from attaching metadata to an agent's identity or redirecting a valid statement to other content. A statement only proves that the agent published the resource; it does not vouch for the truth of the resource's contents.

13.8. Confusable Names

Agent URIs can be visually similar (e.g., examp1e.com). User interfaces that display agent identities **SHOULD** display the A-label form of the host, and **SHOULD** warn when an agent URI or DID is new to the user.

14. Privacy Considerations

The considerations of [RFC6973] apply.

- **Linkability is the point, for agents.** Request signatures make an agent's traffic attributable and linkable across services. That is intended for the agent; it must not extend to the people it acts for. Agent URIs, WIT `jti` values, and `Signature-Agent` values **MUST NOT** encode user identifiers. WIT `jti` values **MUST** be random and not reused across users.
- **Per-user agents.** Deployments that create one agent per end user (e.g., `agent://example.com/u/alice`) make that user's activity linkable across all services the agent visits. Such deployments **SHOULD** use a shared agent identity and convey user context only through authorization tokens sent to parties entitled to it.
- **Enumeration.** API catalogs (Section 11) let anyone list a domain's agents. Domains that do not want their agents to be enumerable should not publish them in a catalog; AIDs remain retrievable by those who already know the agent URI.
- **Verifier fetches.** Fetching an AID reveals to the operator that a verifier has seen the agent. Caching reduces this signal.

15. IANA Considerations

15.1. URI Scheme Registration

This document requests registration of the `agent` URI scheme in the "Uniform Resource Identifier (URI) Schemes" registry per [RFC7595]:

- Scheme name: agent
- Status: Permanent
- Applications/protocols that use this scheme name: Identification of software agents and retrieval of their Agent Identity Documents.
- Contact: The authors.
- Change controller: IETF.

- Reference: Section 4 of this document.

This document does not define or request registration of agent+<suffix> schemes.

15.2. Well-Known URI Registration

This document requests registration in the "Well-Known URIs" registry [RFC8615]:

- URI suffix: agent
- Change controller: IETF
- Specification document: Section 4.3 of this document
- Status: permanent
- Related information: Sub-paths are used; see Section 4.3.

15.3. Media Types

This document requests registration of the following media types per [RFC6838]:

- application/agent-id+json, for Agent Identity Documents (Section 5).
- application/agent-statement+jwt, for Agent Statements (Section 7). The +jwt structured syntax suffix is registered by [RFC8417].

Full registration templates will be provided in a future revision.

15.4. Signature-Agent Type

Section 8 uses the Signature-Agent member parameter value type=agent-id. [WEB-BOT-AUTH] does not currently establish a registry of type values. This document asks that such a registry be created and that agent-id be registered in it, referencing Section 8 of this document (see Appendix B).

This document does not register any HTTP field names.

15.5. Link Relations

This document requests registration of agent-identity in the "Link Relation Types" registry [RFC8288], with description "Refers to the Agent Identity Document of the link context." Registration of agent-card, mcp-server-card, and oauth-client-metadata is an open issue (Appendix B).

15.6. OAuth Client Metadata

This document requests registration of agent_uri in the "OAuth Dynamic Client Registration Metadata" registry [RFC7591], with description "Canonical agent URI of the client".

15.7. JSON Web Key Parameters

This document requests registration of agent_use in the "JSON Web Key Parameters" registry [RFC7517], with description "Purposes for which an Agent Key may be used", parameter information class "Public", used with all key types, referencing Section 5.3 of this document.

15.8. JSON Web Token Claims

This document requests registration of resource_digest in the "JSON Web Token Claims" registry [RFC7519], with description "Digest of the resource identified by the sub claim", referencing Section 7 of this document.

16. References

16.1. Normative References

[CID] Sporny, M. and M. Jones, "Controlled Identifiers v1.0", W3C Recommendation, May 2025, <https://www.w3.org/TR/cid-1.0/>.

[DID-CORE] Sporny, M., Longley, D., Sabadello, M., Reed, D., Steele, O., and C. Allen, "Decentralized Identifiers (DIDs) v1.0", W3C Recommendation, July 2022, <https://www.w3.org/TR/did-core/>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/rfc/rfc2119>.

[RFC3339] Klyne, G. and C. Newman, "Date and Time on the Internet: Timestamps", RFC 3339, DOI 10.17487/RFC3339, July 2002, <https://www.rfc-editor.org/rfc/rfc3339>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/rfc/rfc3986>.

[RFC5234] Crocker, D., Ed. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, RFC 5234, DOI 10.17487/RFC5234, January 2008, <https://www.rfc-editor.org/rfc/rfc5234>.

[RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", RFC 5890, DOI 10.17487/RFC5890, August 2010, <https://www.rfc-editor.org/rfc/rfc5890>.

[RFC5891] Klensin, J., "Internationalized Domain Names in Applications (IDNA): Protocol", RFC 5891, DOI 10.17487/RFC5891, August 2010, <https://www.rfc-editor.org/rfc/rfc5891>.

[RFC6838] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", BCP 13, RFC 6838, DOI 10.17487/RFC6838, January 2013, <https://www.rfc-editor.org/rfc/rfc6838>.

[RFC6890] Cotton, M., Vegoda, L., Bonica, R., Ed., and B. Haberman, "Special-Purpose IP Address Registries", BCP 153, RFC 6890, DOI 10.17487/RFC6890, April 2013, <https://www.rfc-editor.org/rfc/rfc6890>.

[RFC7515] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Signature (JWS)", RFC 7515, DOI 10.17487/RFC7515, May 2015, <https://www.rfc-editor.org/rfc/rfc7515>.

[RFC7517] Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <https://www.rfc-editor.org/rfc/rfc7517>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/rfc/rfc7519>.

[RFC7591] Richer, J., Ed., Jones, M., Bradley, J., Machulak, M., and P. Hunt, "OAuth 2.0 Dynamic Client Registration Protocol", RFC 7591, DOI 10.17487/RFC7591, July 2015, <https://www.rfc-editor.org/rfc/rfc7591>.

[RFC7595] Thaler, D., Ed., Hansen, T., and T. Hardie, "Guidelines and Registration Procedures for URI Schemes", BCP 35, RFC 7595, DOI 10.17487/RFC7595, June 2015, <https://www.rfc-editor.org/rfc/rfc7595>.

[RFC7638] Jones, M. and N. Sakimura, "JSON Web Key (JWK) Thumbprint", RFC 7638, DOI 10.17487/RFC7638, September 2015, <https://www.rfc-editor.org/rfc/rfc7638>.

[RFC7800] Jones, M., Bradley, J., and H. Tschofenig, "Proof-of-Possession Key Semantics for JSON Web Tokens (JWTs)", RFC 7800, DOI 10.17487/RFC7800, April 2016, <https://www.rfc-editor.org/rfc/rfc7800>.

[RFC8037] Liusvaara, I., "CFRG Elliptic Curve Diffie-Hellman (ECDH) and Signatures in JSON Object Signing and Encryption (JOSE)", RFC 8037, DOI 10.17487/RFC8037, January 2017, <https://www.rfc-editor.org/rfc/rfc8037>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/rfc/rfc8174>.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/rfc/rfc8259>.

[RFC8288] Nottingham, M., "Web Linking", RFC 8288, DOI 10.17487/RFC8288, October 2017, <https://www.rfc-editor.org/rfc/rfc8288>.

[RFC8615] Nottingham, M., "Well-Known Uniform Resource Identifiers (URIs)", RFC 8615, DOI 10.17487/RFC8615, May 2019, <https://www.rfc-editor.org/rfc/rfc8615>.

[RFC8693] Jones, M., Nadalin, A., Campbell, B., Ed., Bradley, J., and C. Mortimore, "OAuth 2.0 Token Exchange", RFC 8693, DOI 10.17487/RFC8693, January 2020, <https://www.rfc-editor.org/rfc/rfc8693>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/rfc/rfc8725>.

[RFC8785] Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <https://www.rfc-editor.org/rfc/rfc8785>.

[RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <https://www.rfc-editor.org/rfc/rfc9110>.

[RFC9111] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Caching", STD 98, RFC 9111, DOI 10.17487/RFC9111, June 2022, <https://www.rfc-editor.org/rfc/rfc9111>.

[RFC9421] Backman, A., Ed., Richer, J., Ed., and M. Sporny, "HTTP Message Signatures", RFC 9421, DOI 10.17487/RFC9421, February 2024, <https://www.rfc-editor.org/rfc/rfc9421>.

[RFC9530] Polli, R. and L. Pardue, "Digest Fields", RFC 9530, DOI 10.17487/RFC9530, February 2024, <https://www.rfc-editor.org/rfc/rfc9530>.

[RFC9651] Nottingham, M. and P. Kamp, "Structured Field Values for HTTP", RFC 9651, DOI 10.17487/RFC9651, September 2024, <https://www.rfc-editor.org/rfc/rfc9651>.

[WEB-BOT-AUTH] Meunier, T. and S. Major, "HTTP Message Signatures for automated traffic", Work in Progress, Internet-Draft, draft-ietf-webbotauth-httpsig-protocol-00, 1 September 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-webbotauth-httpsig-protocol-00>.

[WIMSE-CREDS] Campbell, B., Salowey, J. A., Schwenkschuster, A., Sheffer, Y., and Y. Rosomakho, "WIMSE Workload Credentials", Work in Progress, Internet-Draft, draft-ietf-wimse-workload-creds-02, 2 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-wimse-workload-creds-02>.

16.2. Informative References

[A2A] A2A Project, Linux Foundation, "Agent2Agent (A2A) Protocol Specification", 2025, <https://a2a-protocol.org/latest/specification/>.

[DID-WEBVH] Decentralized Identity Foundation, "The did:webvh DID Method", 2025, <https://identity.foundation/didwebvh/>.

[MCP] Model Context Protocol Project, "Model Context Protocol Specification", 2025, <https://modelcontextprotocol.io/specification/>.

[OAUTH-CIMD] Parecki, A. and E. Smith, "OAuth Client ID Metadata Document", Work in Progress, Internet-Draft, draft-ietf-oauth-client-id-metadata-document-02, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-client-id-metadata-document-02>.

[RFC6973] Cooper, A., Tschofenig, H., Aboba, B., Peterson, J., Morris, J., Hansen, M., and R. Smith, "Privacy Considerations for Internet Protocols", RFC 6973, DOI 10.17487/RFC6973, July 2013, <https://www.rfc-editor.org/rfc/rfc6973>.

[RFC8417] Hunt, P., Ed., Jones, M., Denniss, W., and M. Ansari, "Security Event Token (SET)", RFC 8417, DOI 10.17487/RFC8417, July 2018, <https://www.rfc-editor.org/rfc/rfc8417>.

[RFC9126] Lodderstedt, T., Campbell, B., Sakimura, N., Tonge, D., and F. Skokan, "OAuth 2.0 Pushed Authorization Requests", RFC 9126, DOI 10.17487/RFC9126, September 2021, <https://www.rfc-editor.org/rfc/rfc9126>.

[RFC9264] Wilde, E. and H. Van de Sompel, "Linkset: Media Types and a Link Relation Type for Link Sets", RFC 9264, DOI 10.17487/RFC9264, July 2022, <https://www.rfc-editor.org/rfc/rfc9264>.

[RFC9449] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)", RFC 9449, DOI 10.17487/RFC9449, September 2023, <https://www.rfc-editor.org/rfc/rfc9449>.

[RFC9727] Smith, K., "api-catalog: A Well-Known URI and Link Relation to Help Discovery of APIs", RFC 9727, DOI 10.17487/RFC9727, June 2025, <https://www.rfc-editor.org/rfc/rfc9727>.

[RFC9728] Jones, M.B., Hunt, P., and A. Parecki, "OAuth 2.0 Protected Resource Metadata", RFC 9728, DOI 10.17487/RFC9728, April 2025, <https://www.rfc-editor.org/rfc/rfc9728>.

[RFC9943] Birkholz, H., Delignat-Lavaud, A., Fournet, C., Deshpande, Y., and S. Lasker, "An Architecture for Trustworthy and Transparent Digital Supply Chains", RFC 9943, DOI 10.17487/RFC9943, June 2026, <https://www.rfc-editor.org/rfc/rfc9943>.

[TXN-TOKENS] Tulshibagwale, A., Fletcher, G., and P. Kasselman, "Transaction Tokens", Work in Progress, Internet-Draft, draft-ietf-oauth-transaction-tokens-11, 30 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-transaction-tokens-11>.

[WIMSE-HTTPSIG] Salowey, J. A. and Y. Sheffer, "WIMSE Workload-to-Workload Authentication with HTTP Signatures", Work in Progress, Internet-Draft, draft-ietf-wimse-http-signature-08, 20 October 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-wimse-http-signature-08>.

Appendix A. Test Vectors: Agent URI Processing

This appendix is informative. It gives expected results for the syntactic processing defined in Section 4 and Section 5.4, so that independent implementations can check that they agree. Vectors for statements, request signatures, and Workload Identity Tokens will be added once an implementation can produce and cross-check them (Appendix B).

A.1. Parsing and Canonicalization

Each vector shows an input, then its result. "valid" inputs are canonical. "normalized" inputs differ from the ABNF only by deviations listed in Section 4.2; parsers **MAY** accept them and, if they do, **MUST** produce the canonical form shown. "reject" inputs **MUST** be rejected. <U+00FC> denotes the single character LATIN SMALL LETTER U WITH DIAERESIS.

```
agent://example.com/planner
   valid       agent://example.com/planner
agent://planner.example.com
   valid       agent://planner.example.com    (root agent)
agent://example.com:8443/planner
   valid       agent://example.com:8443/planner
agent://example.com/acme/billing-assistant
   valid       agent://example.com/acme/billing-assistant
agent://123.example/planner
   valid       agent://123.example/planner
agent://example.com/p#kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k
   valid       agent://example.com/p          (plus key fragment)

AGENT://Example.COM/planner
   normalized  agent://example.com/planner
agent://example.com:443/planner
   normalized  agent://example.com/planner
agent://example.com:/planner
   normalized  agent://example.com/planner
agent://example.com/planner/
   normalized  agent://example.com/planner
agent://planner.example.com/
   normalized  agent://planner.example.com
agent://b<U+00FC>cher.example/planner
   normalized  agent://xn--bcher-kva.example/planner

agent://192.168.0.1/planner             reject  all-digit final label
agent://localhost/planner               reject  single-label host
agent://[::1]/planner                   reject  IP literal
agent://user@example.com/planner        reject  userinfo
agent://example.com./planner            reject  trailing "." on host
agent://example.com:0443/planner        reject  leading zero in port
agent://example.com:65536/planner       reject  port out of range
agent://example.com/planner?task=1      reject  query
agent://example.com/Planner             reject  uppercase name-path
agent://example.com/plan%6Eer           reject  percent-encoding
agent://example.com/a/../planner        reject  dot-segment
agent://example.com//planner            reject  empty segment
agent://example.com/-planner            reject  leading "-"
agent://example.com/_                   reject  reserved segment
agent://example.com/a/b/c/d/e/f/g/h/i   reject  more than 8 segments
agent+https://example.com/planner       reject  not the agent scheme
```

Figure 7: Parsing and canonicalization vectors

`agent://example.com/p` with and without the key fragment identify the same agent (Section 4.2).

A.2. Mapping to AID URLs

```
agent://example.com/planner
   https://example.com/.well-known/agent/planner
agent://planner.example.com
   https://planner.example.com/.well-known/agent/_
agent://example.com:8443/planner
   https://example.com:8443/.well-known/agent/planner
agent://example.com/acme/billing-assistant
   https://example.com/.well-known/agent/acme/billing-assistant
agent://xn--bcher-kva.example/planner
   https://xn--bcher-kva.example/.well-known/agent/planner
```

Figure 8: Agent URI to AID URL vectors

A.3. Reverse Mapping of Signature-Agent Values

Values are compared exactly, without normalization (Section 8.2).

```
https://example.com/.well-known/agent/planner
   agent://example.com/planner
https://planner.example.com/.well-known/agent/_
   agent://planner.example.com
https://example.com:8443/.well-known/agent/planner
   agent://example.com:8443/planner
https://Example.com/.well-known/agent/planner
   reject  not canonical (host case)
https://example.com:443/.well-known/agent/planner
   reject  not canonical (default port)
https://example.com/.well-known/agent/planner/
   reject  not canonical (trailing "/")
https://example.com/.well-known/agent/planner?v=1
   reject  query
http://example.com/.well-known/agent/planner
   reject  not HTTPS
https://example.com/.well-known/agent
   reject  neither a name-path nor the reserved segment
https://example.com/.well-known/agent/_/planner
   reject  reserved segment inside a name-path
https://example.com/.well-known/agent/Planner
   reject  invalid name-path
https://example.com/.well-known/agent/planner#k
   reject  fragment
https://b<U+00FC>cher.example/.well-known/agent/planner
   reject  not canonical (U-label host)
```

Figure 9: Reverse mapping vectors

A.4. AID Origin Comparison

For agent://example.com/planner, the AID Origin is ("https", "example.com", 443). Link targets compare as follows (Section 5.4):

```
https://example.com/planner/agent-card.json          same origin
https://example.com:443/planner/agent-card.json      same origin
https://EXAMPLE.com/planner/agent-card.json          same origin
https://planner.example.com/agent-card.json          different host
https://example.com:8443/planner/agent-card.json     different port
https://example.com./planner/agent-card.json         different host
http://example.com/planner/agent-card.json           invalid href
```

Figure 10: AID Origin comparison vectors

A link whose `href` is not an HTTPS URL is invalid regardless of origin, and a statement cannot make it valid (Section 5.4).

Appendix B. Open Issues

This appendix will be removed before publication.

1. **Signature-Agent type registry.** [WEB-BOT-AUTH] has no registry for `type` values. Work with the webbotauth WG to create one, or to define `agent-id` there. Until then, verifiers that only implement [WEB-BOT-AUTH] ignore agent identities.
2. **WIMSE alignment.** Discuss with the WIMSE WG whether port-free agent URIs are acceptable workload identifiers (their authority fits the trust domain shape of WIMSE identifiers), whether AID-based issuer discovery can become a recognized alternative to out-of-band trust-anchor provisioning, and whether instances should also be able to use the WIMSE HTTP signature profile [WIMSE-HTTPSIG].
3. **Back-references from protocol metadata.** Agree with the A2A and MCP projects on how an Agent Card or MCP server card names its agent URI, and who registers the `agent-card` and `mcp-server-card` link relations.
4. **Retired keys.** Consider an optional list of retired keys with retirement times, so that statements issued before a planned rotation remain verifiable without re-issuance, while compromised keys are still removed.
5. **Reserved root segment.** `/.well-known/agent/_` is chosen to work on static hosting. Alternatives (a separate well-known suffix for the root agent) should be evaluated.
6. **Name path depth and character set.** Lowercase LDH segments, at most 8, are chosen for unambiguous comparison. Confirm this is sufficient for multi-tenant platforms.
7. **DNS-based verification.** Consider an alternative proof of control via a DNS TXT record, for operators that cannot serve well-known paths.
8. **Port.** Confirm that non-default ports should be permitted in agent identities at all, given they complicate certificate and policy handling and are excluded from the agent-instance profile (Section 9).
9. **Cryptographic test vectors.** Appendix A covers URI processing. Once an implementation can produce and cross-check them, add vectors for: statement `iss`/`sub` binding; DID key conversion and per-purpose selection; `agent_use` enforcement, including duplicate key material and malformed values; a preserved DID with substituted same-origin client metadata keys; a DID-anchored client using `jwks_uri` with `private_key_jwt` (not attributed); an origin-anchored client whose statement-covered metadata points to a `jwks_uri` off the AID Origin (not attributed); an attacker-owned client metadata document claiming an unrelated origin-anchored agent's `agent_uri`,

both with only a client assertion (not attributed) and with a request signed by the attacker's own agent (attributed to the attacker's agent, never the claimed one); a WIT whose sub has a port; a WIT with a future iat; content tampering with a valid signature; fetches to forbidden destinations; and expired credentials. Two independent verifiers should reach the same decision for each.

Appendix C. Changes from draft-narvaneni-agent-uri-03

This revision changes the scope of the document from "addressing and discovery" to "identity".

Removed:

- `agent+<protocol>` transport-binding schemes and the Transport Bindings registry.
- Invocation semantics (GET/POST rules, fallback behaviour), query parameters in agent URIs, and conformance levels.
- The /.well-known/agents.json registry, replaced by the deterministic mapping of Section 4.3 and optional RFC 9727 catalogs.
- The agent descriptor's capability fields (skills, transport, interactionModel, versioning, content negotiation). Protocol metadata documents are linked instead.
- DID authorities in agent URIs, replaced by DID anchoring (Section 6.2).
- Cross-cutting conventions (tracing, idempotency, rate limits, health) and the Interaction Models registry.
- The monotonic scope narrowing rule, which conflicted with [RFC8693].

Added:

- Strict URI syntax, normalization, comparison, and reverse-mapping rules.
- The Agent Identity Document, a single Key Identifier scheme, key purposes (including agent_use for origin-anchored keys), and origin and DID anchoring with DID pinning.
- Agent Statements bound to the links that carry them.
- A request-signing profile of [WEB-BOT-AUTH], and an agent-instance profile reusing the WIMSE Workload Identity Token format with AID-based trust.
- OAuth bindings that separate metadata association from agent attribution, define which client keys qualify, and describe verified identity at consent using Pushed Authorization Requests.
- Security and privacy considerations specific to agent identity, including a fetch policy covering every verifier-initiated request.
- Test vectors for URI processing (Appendix A).

Acknowledgments

The authors thank the reviewers of earlier revisions, whose feedback led to the change of scope in this revision.

Authors' Addresses

Yaswanth Narvaneni
Independent Researcher
London
United Kingdom
Email: yaswanth+ietf@gmail.com

Sai Purnima Ravi
Independent Researcher
Email: r.sai.purnima+ietf@gmail.com